

A Decision Procedure for Path Feasibility of String Manipulating Programs with Integer Data Type

Taolue Chen¹, Matthew Hague², Jinlong He³, Denghang Hu³,
Anthony Widjaja Lin⁴, Philipp Rümmer⁵, and Zhilin Wu³

¹ Birkbeck, University of London, UK

² Royal Holloway, University of London, UK

³ Institute of Software, Chinese Academy of Sciences, China

⁴ Technical University of Kaiserslautern, Germany

⁵ Uppsala University, Sweden

Abstract. Strings are widely used in programs, especially in web applications. Integer data type occurs naturally in string-manipulating programs, and is frequently used to refer to lengths of, or positions in, strings. Analysis and testing of string-manipulating programs can be formulated as the path feasibility problem: given a symbolic execution path, does there exist an assignment to the inputs that yields a concrete execution that realizes this path? Such a problem can naturally be reformulated as a string constraint solving problem. Although state-of-the-art string constraint solvers usually provide support for both string and integer data types, they mainly resort to heuristics without completeness guarantees.

In this paper, we propose a decision procedure for a class of string-manipulating programs which includes not only a wide range of string operations such as concatenation, replaceAll, reverse, and finite transducers, but also those involving the integer data-type such as length, indexOf, and substring. To the best of our knowledge, this represents one of the most expressive string constraint languages that is currently known to be decidable. Our decision procedure is based on a variant of cost register automata. We implement the decision procedure, giving rise to a new solver OSTRICH+. We evaluate the performance of OSTRICH+ on a wide range of existing and new benchmarks. The experimental results show that OSTRICH+ is the first string decision procedure capable of tackling finite transducers and integer constraints, whilst its overall performance is comparable with the state-of-the-art string constraint solvers.

1 Introduction

String-manipulating programs are notoriously subtle, and their potential bugs may bring severe security consequences. A typical example is cross-site scripting (XSS), which is among the OWASP Top 10 Application Security Risks [?]. Integer data type occurs naturally and extensively in string-manipulating programs. An effective and increasingly popular method for identifying bugs, including XSS, is symbolic execution [?]. In a nutshell, this technique analyses static paths

through the program being considered. Each of these paths can be viewed as a constraint φ over appropriate data domains, and symbolic execution tools demand fast constraint solvers to check the satisfiability of φ . Such constraint solvers need to support all data-type operations occurring in a program.

Typically, mainstream programming languages provide standard string functions such as concatenation, **replace**, and **replaceAll**. Moreover, Web programming languages usually provide complex string operations (e.g. `htmlEscape` and `trim`), which are conveniently modelled as finite transducers, to sanitise malicious user inputs [?]. Nevertheless, apart from these operations involving only the string data type, functions such as **length**, **indexOf**, and **substring**, which can convert strings to integers and vice versa, are also heavily used in practice; for instance, it was reported [?] that **length**, **indexOf**, **substring**, and variants thereof, comprise over 80% of string function occurrences in 18 popular JavaScript applications, notably outnumbering concatenation. The introduction of integers exacerbates the intricacy of string-manipulating programs, and poses new theoretical and practical challenges in solver development.

When combining strings and integers, decidability can easily be lost; for instance, the string theory with concatenation and letter counting functions is undecidable [?,?]. Remarkably, it is still a major open problem whether the string theory with concatenation (arguably the simplest string operation) and length function (arguably the most common string-number function) is decidable [?]. One promising approach to retain decidability is to enforce a syntactic restriction to the constraints. In the literature, these restrictions include solved forms [?], acyclicity [?,?], and straight-line fragment (aka programs in single static assignment form) [?,?]. On the one hand, such a restriction has led to decidability of string constraint solving with complex string operations (not only concatenation, but also finite transducers) and integer operations (letter-counting, **length**, **indexOf**, etc.); see, e.g., [?]. On the other hand, there is a lot of evidence (e.g. from benchmark) that many practical string constraints do satisfy such syntactic restrictions.

Approaches to building practical string solvers could essentially be classified into two categories. Firstly, one could support as many constraints as possible, but primarily resort to heuristics, offering no completeness/termination guarantee. This is a realistic approach since, as mentioned above, the problem involving both string and integer data types is in general undecidable. Many solvers belong to this category, e.g., CVC4 [?], Z3 [?,?], Z3-str3 [?], S3(P) [?,?], Trau [?] (or its variants Trau+ [?] and Z3-Trau [?]), ABC [?], and Slent [?]. Completeness guarantees are, however, valuable since the performance of heuristics can be difficult to predict. The second approach is to develop solvers for decidable fragments supporting both strings and integers (e.g. [?,?]). Solvers in this category include Norn [?], SLOTH [?], and OSTRICH [?]. The fragment without complex string operations (e.g. **replaceAll** and finite transducers, but **length**) can be handled quite well by Norn. The fragment without length constraints (but **replaceAll** and finite transducers) can be handled effectively by OSTRICH and SLOTH. Moreover, most existing solvers that belong to the first category do

not support complex string operations like **replaceAll** and finite transducers as well. This motivates the following problem: provide a decision procedure that supports both string and integer data type, with completeness guarantee and meanwhile admitting efficient implementation.

We argue that this problem is highly challenging. A deeper examination of the algorithms used by OSTRICH and SLOTH reveals that, unlike the case for Norn, it would not be straightforward to extend OSTRICH and SLOTH with integer constraints. First and foremost, the complexity of the fragment used by Norn (i.e. without transducers and **replaceAll**) is solvable in exponential time, even in the presence of integer constraints. This is not the case for the straight-line fragments with transducers/**replaceAll**, which require at least double exponential time (regardless of the integer constraints). This unfortunately manifests itself in the size of symbolic representations of the solutions. SLOTH [?] computes a representation of all solutions “eagerly” as (alternating) finite transducers. Dealing with integer data type requires to compute the Parikh images of these transducers [?], which would result in a quantifier-free linear integer arithmetic formula (LIA for short) of double exponential size, thus giving us a triple exponential time algorithm, since LIA formulas are solved in exponential time (see e.g. [?]). Lin and Barcelo [?] provided a double exponential upper bound in the length of the strings in the solution, and showed that the double exponential time theoretical complexity could be retained. This, however, does not result in a practical algorithm since it requires all strings of double exponential size to be enumerated. OSTRICH [?] adopted a “lazy” approach and computed the pre-images of regular languages step by step, which is more scalable than the “eager” approach adopted by SLOTH and results in a highly competitive solver. It uses recognisable relations (a finite union of products of regular languages) as symbolic representations. Nevertheless, extending this approach to integer constraints is not obvious since integer constraints break the independence between different string variables in the recognisable relations.

Contribution. We provide a decision procedure for an expressive class of string constraints involving the integer data type, which includes not only concatenation, **replace/replaceAll**, **reverse**, finite transducers, and regular constraints, but also **length**, **indexOf** and **substring**. The decision procedure utilizes a variant of cost-register automata introduced by Alur et al. [?], which are called cost-enriched finite automata (CEFA) for convenience. Intuitively, each CEFA records the connection between a string variable and its associated integer variables. With CEFAs, the concept of recognisable relations is then naturally extended to accommodate integers. The integer constraints, however, are detached from CEFAs rather than being part of CEFAs. This allows to preserve the independence of string variables in the recognisable relation. The crux of the decision procedure is to compute the backward images of CEFAs under string functions, where each cost register (integer variable) might be split into several ones, thus extending but still in the same flavour as OSTRICH for string constraints without the integer data type [?]. Such an approach is able to treat a wide range of string functions in a generic, and yet simple, way. To the best of our knowledge,

the class of string constraints considered in this paper is currently one of the most expressive string theories involving the integer data type known to enjoy a decision procedure.

We implement the decision procedure based on the recent OSTRICH solver [?], resulting in OSTRICH+. We perform experiments on a wide range of benchmark suites, including those where both **replace/replaceAll**/finite transducers and **length/indexOf/substring** occur, as well as the well-known benchmarks Kaluza and PyEx. The results show that 1) OSTRICH+ so far is the only string constraint solver capable of dealing with finite transducers and integer constraints, and 2) its overall performance is comparable with the best state-of-the-art string constraint solvers (e.g. CVC4 and Z3-Trau) which are short of completeness guarantees.

The rest of the paper is structured as follows: Section ?? introduces the preliminaries. Section ?? defines the class of string-manipulating programs with integer data type. Section ?? presents the decision procedure. Section ?? presents the benchmarks and experiments for the evaluation. The paper is concluded in Section ?. Missing proofs, implementation details and further examples can be found in the appendix.

2 Preliminaries

We write $\mathbb{N}$ and $\mathbb{Z}$ for the sets of natural and integer numbers, respectively. For $n \in \mathbb{N}$ with $n \geq 1$, $[n]$ denotes $\{1, \dots, n\}$; for $m, n \in \mathbb{N}$ with $m \leq n$, $[m, n]$ denotes $\{i \in \mathbb{N} \mid m \leq i \leq n\}$. Throughout the paper, Σ is a finite alphabet, ranged over by $a, b, \dots$.

Strings, languages, and transductions. A string over Σ is a (possibly empty) sequence of elements from Σ , denoted by $u, v, w, \dots$. An empty string is denoted by ε . We write Σ^* (resp., Σ^+) for the set of all (resp. nonempty) strings over Σ . For a string u , we use $|u|$ to denote the number of letters in u . In particular, $|\varepsilon| = 0$. Moreover, for $a \in \Sigma$, let $|u|_a$ denote the number of occurrences of a in u . Assume $u = a_0 \cdots a_{n-1}$ is nonempty and $i < j \in [0, n-1]$. We let $u[i]$ denote a_i and $u[i, j]$ for the substring $a_i \cdots a_j$.

Let u, v be two strings. We use $u \cdot v$ to denote the concatenation of u and v . The string u is said to be a prefix of v if $v = u \cdot v'$ for some string v' . In addition, if $u \neq v$, then u is said to be a strict prefix of v . If $v = u \cdot v'$ for some string v' , then we use $u^{-1}v$ to denote v' . In particular, $\varepsilon^{-1}v = v$. If $u = a_0 \cdots a_{n-1}$ is nonempty, then we use $u^{(r)}$ to denote the reverse of u , that is, $u^{(r)} = a_{n-1} \cdots a_0$.

A transduction over Σ is a binary relation over Σ^* , namely, a subset of $\Sigma^* \times \Sigma^*$. We will use $T_1, T_2, \dots$ to denote transductions. For two transductions T_1 and T_2 , we will use $T_1 \cdot T_2$ to denote the composition of T_1 and T_2 , namely, $T_1 \cdot T_2 = \{(u, w) \in \Sigma^* \times \Sigma^* \mid \text{there exists } v \in \Sigma^* \text{ s.t. } (u, v) \in T_1 \text{ and } (v, w) \in T_2\}$.

Recognisable relations. We assume familiarity with standard regular language. Recall that a regular language L can be represented by a regular expression $e \in \text{RegExp}$ whereby we usually write $L = \mathcal{L}(e)$.

Intuitively, a recognisable relation is simply a finite union of Cartesian products of regular languages. Formally, an r -ary relation $R \subseteq \Sigma^* \times \dots \times \Sigma^*$ is recognisable if $R = \bigcup_{i=1}^n L_1^{(i)} \times \dots \times L_r^{(i)}$ where $L_j^{(i)}$ is regular for each $j \in [r]$. A representation of a recognisable relation $R = \bigcup_{i=1}^n L_1^{(i)} \times \dots \times L_r^{(i)}$ is $(\mathcal{A}_1^{(i)}, \dots, \mathcal{A}_r^{(i)})_{1 \leq i \leq n}$ such that each $\mathcal{A}_j^{(i)}$ is an NFA with $\mathcal{L}(\mathcal{A}_j^{(i)}) = L_j^{(i)}$. The tuples $(\mathcal{A}_1^{(i)}, \dots, \mathcal{A}_r^{(i)})$ are called the disjuncts of the representation and the NFAs $\mathcal{A}_j^{(i)}$ are called the atoms of the representation.

Automata models. A (nondeterministic) finite automaton (NFA) is a tuple $\mathcal{A} = (Q, \Sigma, \delta, I, F)$, where Q is a finite set of states, Σ is a finite alphabet, $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation, $I, F \subseteq Q$ are the set of initial and final states respectively. For readability, we write a transition $(q, a, q') \in \delta$ as $q \xrightarrow[a]{a} q'$ (or simply $q \xrightarrow{a} q'$). The size of an NFA $\mathcal{A}$, denoted by $|\mathcal{A}|$, is defined as the number of transitions of $\mathcal{A}$. A run of $\mathcal{A}$ on a string $w = a_1 \dots a_n$ is a sequence of transitions $q_0 \xrightarrow{a_1} q_1 \dots q_{n-1} \xrightarrow{a_n} q_n$ with $q_0 \in I$. The run is accepting if $q_n \in F$. A string w is accepted by an NFA $\mathcal{A}$ if there is an accepting run of $\mathcal{A}$ on w . In particular, the empty string ε is accepted by $\mathcal{A}$ if $I \cap F \neq \emptyset$. The language of $\mathcal{A}$, denoted by $\mathcal{L}(\mathcal{A})$, is the set of strings accepted by $\mathcal{A}$. An NFA $\mathcal{A}$ is said to be deterministic if I is a singleton and, for every $q \in Q$ and $a \in \Sigma$, there is at most one state $q' \in Q$ such that $(q, a, q') \in \delta$. It is well-known that finite automata capture regular languages precisely.

A nondeterministic finite transducer (NFT) $\mathcal{T}$ is an extension of NFA with outputs. Formally, an NFT $\mathcal{T}$ is a tuple $(Q, \Sigma, \delta, I, F)$, where Q, Σ, I, F are as in NFA and the transition relation δ is a finite subset of $Q \times \Sigma \times Q \times \Sigma^*$. Similarly to NFA, for readability, we write a transition $(q, a, q', u) \in \delta$ as $q \xrightarrow[\delta]{a, u} q'$ or $q \xrightarrow{a, u} q'$. The size of an NFT $\mathcal{T}$, denoted by $|\mathcal{T}|$, is defined as the sum of the sizes of the transitions of $\mathcal{T}$, where the size of a transition $q \xrightarrow{a, u} q'$ is defined as $|u|+3$. A run of $\mathcal{T}$ over a string $w = a_1 \dots a_n$ is a sequence of transitions $q_0 \xrightarrow{a_1, u_1} q_1 \dots q_{n-1} \xrightarrow{a_n, u_n} q_n$ with $q_0 \in I$. The run is accepting if $q_n \in F$. The string $u_1 \dots u_n$ is called the output of the run. The transduction defined by $\mathcal{T}$, denoted by $\mathcal{T}(\mathcal{T})$, is the set of string pairs (w, u) such that there is an accepting run of $\mathcal{T}$ on w , with the output u . An NFT $\mathcal{T}$ is said to be deterministic if I is a singleton, and, for every $q \in Q$ and $a \in \Sigma$ there is at most one pair $(q', u) \in Q \times \Sigma^*$ such that $(q, a, q', u) \in \delta$. In this paper, we are primarily interested in functional finite transducers (FFT), i.e., finite transducers that define functions instead of relations. (For instance, deterministic finite transducers are always functional.)

We will also use standard quantifier-free/existential linear integer arithmetic (LIA) formulae, which are typically ranged over by ϕ, φ , etc.

3 String-Manipulating Programs with Integer Data Type

In this paper, we consider logics involving two data-types, i.e., the string data-type and the integer data-type. As a convention, $u, v, \dots$ denote string constants,

$c, d, \dots$ denote integer constants, $x, y, \dots$ denote string variables, and $i, j, \dots$ denote integer variables.

We consider symbolic execution of string-manipulating programs with numeric conditions (abbreviated as SL_{int}), defined by the following rules,

$$\begin{aligned} S &::= x := y \cdot z \mid x := \text{replaceAll}_{e,u}(y) \mid x := \text{reverse}(y) \mid x := \mathcal{T}(y) \mid \\ &\quad x := \text{substring}(y, t_1, t_2) \mid \text{assert}(\varphi) \mid S; S, \\ \varphi &::= x \in \mathcal{A} \mid t_1 \ o \ t_2 \mid \varphi \vee \varphi \mid \varphi \wedge \varphi, \end{aligned}$$

where e is a regular expression over Σ , $u \in \Sigma^*$, $\mathcal{T}$ is an FFT, $\mathcal{A}$ is an NFA, $o \in \{=, \neq, \geq, \leq, >, <\}$, and t_1, t_2 are integer terms defined by the following rules,

$$t ::= i \mid c \mid \text{length}(x) \mid \text{indexOf}_v(x, i) \mid ct \mid t + t, \text{ where } c \in \mathbb{Z}, v \in \Sigma^+.$$

We require that the string-manipulating programs are in single static assignment (SSA) form. Note that SSA form imposes restrictions only on the assignment statements, but not on the assertions. A string variable x in an SL_{int} program S is called an input string variable of S if it does not appear on the left-hand side of the assignment statements of S . A variable in S is called an input variable if it is either an input string variable or an integer variable.

Semantics. The semantics of SL_{int} is explained as follows.

- The assignment $x := y \cdot z$ denotes that x is the concatenation of two strings y and z .
- The assignment $x := \text{replaceAll}_{e,u}(y)$ denotes that x is the string obtained by replacing all occurrences of e in y with u , where the leftmost and longest matching of e is used. For instance, $\text{replaceAll}_{(ab)^+,c}(aababaab) = ac \cdot \text{replaceAll}_{(ab)^+,c}(aab) = acac$, since the leftmost and longest matching of $(ab)^+$ in $aababaab$ is $abab$. Here we require that the language defined by e does not contain the empty string, in order to avoid the troublesome definition of the semantics of the matching of the empty string. The formal semantics of the replaceAll function can be found in [?].
- The assignment $x := \text{reverse}(y)$ denotes that x is the reverse of y .
- The assignment $x := \mathcal{T}(y)$ denotes that $(y, x) \in \mathcal{T}(\mathcal{T})$.
- The assignment $x := \text{substring}(y, t_1, t_2)$ denotes that x is equal to the return value of $\text{substring}(y, t_1, t_2)$, where

$$\text{substring}(y, t_1, t_2) = \begin{cases} \epsilon & \text{if } t_1 < 0 \vee t_1 \geq |y| \vee t_2 = 0 \\ y[t_1, \min\{t_1 + t_2 - 1, |y| - 1\}] & \text{o/w} \end{cases}$$

For instance, $\text{substring}(abaab, -1, 1) = \epsilon$, $\text{substring}(abaab, 3, 0) = \epsilon$, $\text{substring}(abaab, 3, 2) = ab$, and $\text{substring}(abaab, 3, 3) = ab$.

- The conditional statement $\text{assert}(x \in \mathcal{A})$ denotes that x belongs to $\mathcal{L}(\mathcal{A})$.
- The conditional statement $\text{assert}(t_1 \ o \ t_2)$ denotes that the value of t_1 is equal to (not equal to, ...) that of t_2 , if $o \in \{=, \neq, \geq, >, \leq, <\}$.
- The integer term $\text{length}(x)$ denotes the length of x .

- The function $\text{indexOf}_v(x, i)$ returns the starting position of the first occurrence of v in x after the position i , if such an occurrence exists, and -1 otherwise. Note that if $i < 0$, then $\text{indexOf}_v(x, i)$ returns $\text{indexOf}_v(x, 0)$, and if $i \geq \text{length}(x)$, then $\text{indexOf}_v(x, i)$ returns -1 . For instance, $\text{indexOf}_{ab}(aaba, -1) = 1$, $\text{indexOf}_{ab}(aaba, 1) = 1$, $\text{indexOf}_{ab}(aaba, 2) = -1$, and $\text{indexOf}_{ab}(aaba, 4) = -1$.

Path feasibility problem. Given an SL_{int} program S , decide whether there are valuations of the input variables so that S can execute to the end.

4 Decision Procedures for Path Feasibility

In this section, we present a decision procedure for the path feasibility problem of SL_{int} . A distinguished feature of the decision procedure is that it conducts backward computation which is lazy and can be done in a modular way. To support this, we extend a regular language with quantitative information of the strings in the language, giving rise to cost-enriched regular languages and corresponding finite automata (Section ??). The crux of the decision procedure is thus to show that the pre-images of cost-enriched regular languages under the string operations in SL_{int} (i.e., concatenation $\cdot$, **replaceAll** _{e, u} , **reverse**, FFTs $\mathcal{T}$, and **substring**) are representable by so called cost-enriched recognisable relations (Section ??). The overall decision procedure is presented in Section ??, supplied by additional complexity analysis.

4.1 Cost-Enriched Regular Languages and Recognisable Relations

Let $k \in \mathbb{N}$ with $k > 0$. A k -cost-enriched string is $(w, (n_1, \dots, n_k))$ where w is a string and $n_i \in \mathbb{Z}$ for all $i \in [k]$. A k -cost-enriched language L is a subset of $\Sigma^* \times \mathbb{Z}^k$. For our purpose, we identify a “regular” fragment of cost-enriched languages as follows.

Definition 1 (Cost-enriched regular languages). Let $k \in \mathbb{N}$ with $k > 0$. A k -cost-enriched language is regular (abbreviated as CERL) if it can be accepted by a cost-enriched finite automaton.

A cost-enriched finite automaton (CEFA) $\mathcal{A}$ is a tuple $(Q, \Sigma, R, \delta, I, F)$ where

- Q, Σ, I, F are defined as in NFAs,
- $R = (r_1, \dots, r_k)$ is a vector of (mutually distinct) cost registers,
- δ is the transition relation which is a finite set of tuples (q, a, q', η) where $q, q' \in Q$, $a \in \Sigma$, and $\eta : R \rightarrow \mathbb{Z}$ is a cost register update function.

For convenience, we usually write $(q, a, q', \eta) \in \delta$ as $q \xrightarrow{a, \eta} q'$.

A run of $\mathcal{A}$ on a k -cost-enriched string $(a_1 \dots a_m, (n_1, \dots, n_k))$ is a transition sequence $q_0 \xrightarrow{a_1, \eta_1} q_1 \dots q_{m-1} \xrightarrow{a_m, \eta_m} q_m$ such that $q_0 \in I$ and $n_i = \sum_{1 \leq j \leq m} \eta_j(r_i)$ for each $i \in [k]$ (Note that the initial values of cost registers are zero). The run is accepting if $q_m \in F$. A k -cost-enriched string $(w, (n_1, \dots, n_k))$ is accepted by $\mathcal{A}$ if there is an accepting run of $\mathcal{A}$ on $(w, (n_1, \dots, n_k))$. In particular, (ε, n) is accepted by $\mathcal{A}$ if $n = 0$ and $I \cap F \neq \emptyset$. The k -cost-enriched language defined by $\mathcal{A}$, denoted by $\mathcal{L}(\mathcal{A})$, is the set of k -cost-enriched strings accepted by $\mathcal{A}$.

The size of a CEFA $\mathcal{A} = (Q, \Sigma, R, \delta, I, F)$, denoted by $|\mathcal{A}|$, is defined as the sum of the sizes of its transitions, where the size of each transition (q, a, q', η) is $\sum_{r \in R} \lceil \log_2(|\eta(r)|) \rceil + 3$. Note here the integer constants in $\mathcal{A}$ are encoded in binary.

Remark 1. CEFAs can be seen as a variant of Cost Register Automata [?], by admitting nondeterminism and discarding partial final cost functions. CEFAs are also closely related to monotonic counter machines [?]. The main difference is that CEFAs discard guards in transitions and allow binary-encoded integers in cost updates, while monotonic counter machines allow guards in transitions but restrict the cost updates to being monotonic and unary, i.e. 0, 1 only. Moreover, we explicitly define CEFAs as language acceptors for cost-enriched languages.

Example 1 (CEFA for **length**). The string function **length** can be captured by CEFAs. For any NFA $\mathcal{A} = (Q, \Sigma, \delta, I, F)$, it is not difficult to see that the cost-enriched language $\{(w, \text{length}(w)) \mid w \in \mathcal{L}(\mathcal{A})\}$ is accepted by a CEFA, i.e., $(Q, \Sigma, (r_1), \delta', I, F)$ such that for each $(q, a, q') \in \delta$, we let $(q, a, q', \eta) \in \delta'$, where $\eta(r_1) = 1$.

For later use, we identify a special $\mathcal{A}_{\text{len}} = (\{q_0\}, \Sigma, (r_1), \{(q_0, a, q_0, \eta) \mid \eta(r_1) = 1\}, \{q_0\}, \{q_0\})$. In other words, $\mathcal{A}_{\text{len}}$ accepts $\{(w, \text{length}(w)) \mid w \in \Sigma^*\}$.

We can show that the function $\text{indexOf}_v(\cdot, \cdot)$ can be captured by a CEFA as well, in the sense that, for any NFA $\mathcal{A}$ and constant string v , we can construct a CEFA $\mathcal{A}_{\text{indexOf}_v}$ accepting $\{(w, (n, \text{indexOf}_v(w, n))) \mid w \in \mathcal{L}(\mathcal{A}), n \leq \text{indexOf}_v(w, n) < |w|\}$. The construction is slightly technical and can be found in Appendix ?? . Note that $\mathcal{A}_{\text{indexOf}_v}$ does not model the corner cases in the semantics of indexOf_v , for instance, $\text{indexOf}_v(w, n) = -1$ if v does not occur after the position n in w .

Given two CEFAs $\mathcal{A}_1 = (Q_1, \Sigma, R_1, \delta_1, I_1, F_1)$ and $\mathcal{A}_2 = (Q_2, \Sigma, \delta_2, R_2, I_2, F_2)$ with $R_1 \cap R_2 = \emptyset$, the product of $\mathcal{A}_1$ and $\mathcal{A}_2$, denoted by $\mathcal{A}_1 \times \mathcal{A}_2$, is defined as $(Q_1 \times Q_2, \Sigma, R_1 \cup R_2, \delta, I_1 \times I_2, F_1 \times F_2)$, where δ comprises the tuples $((q_1, q_2), \sigma, (q'_1, q'_2), \eta)$ such that $(q_1, \sigma, q'_1, \eta_1) \in \delta_1$, $(q_2, \sigma, q'_2, \eta_2) \in \delta_2$, and $\eta = \eta_1 \cup \eta_2$.

For a CEFA $\mathcal{A}$, we use $R(\mathcal{A})$ to denote the vector of cost registers occurring in $\mathcal{A}$. Suppose $\mathcal{A}$ is CEFA with $R(\mathcal{A}) = (r_1, \dots, r_k)$ and $\vec{i} = (i_1, \dots, i_k)$ is a vector of mutually distinct integer variables such that $R(\mathcal{A}) \cap \vec{i} = \emptyset$. We use $\mathcal{A}[\vec{i}/R(\mathcal{A})]$ to denote the CEFA obtained from $\mathcal{A}$ by simultaneously replacing r_j with i_j for $j \in [k]$.

Definition 2 (Cost-enriched recognisable relations). Let $(k_1, \dots, k_l) \in \mathbb{N}^l$ with $k_j > 0$ for every $j \in [l]$. A cost-enriched recognisable relation (CERR) $\mathcal{R} \subseteq (\Sigma^* \times \mathbb{Z}^{k_1}) \times \dots \times (\Sigma^* \times \mathbb{Z}^{k_l})$ is a finite union of products of CERLs. Formally, $\mathcal{R} = \bigcup_{i=1}^n L_{i,1} \times \dots \times L_{i,l}$, where for every $j \in [l]$, $L_{i,j} \subseteq \Sigma^* \times \mathbb{Z}^{k_j}$ is a CERL. A CEFA representation of $\mathcal{R}$ is a collection of CEFA tuples $(\mathcal{A}_{i,1}, \dots, \mathcal{A}_{i,l})_{i \in [n]}$ such that $\mathcal{L}(\mathcal{A}_{i,j}) = L_{i,j}$ for every $i \in [n]$ and $j \in [l]$.

4.2 Pre-images of CERLs under string operations

To unify the presentation, we consider string functions $f : (\Sigma^* \times \mathbb{Z}^{k_1}) \times \cdots \times (\Sigma^* \times \mathbb{Z}^{k_l}) \rightarrow \Sigma^*$. (If there is no integer input parameter, then $k_1, \dots, k_l$ are zero.)

Definition 3 (Cost-enriched pre-images of CERLs). Suppose that $f : (\Sigma^* \times \mathbb{Z}^{k_1}) \times \cdots \times (\Sigma^* \times \mathbb{Z}^{k_l}) \rightarrow \Sigma^*$ is a string function, $L \subseteq \Sigma^* \times \mathbb{Z}^{k_0}$ is a CERL defined by a CEFA $\mathcal{A} = (Q, \Sigma, R, \delta, I, F)$ with $R = (r_1, \dots, r_{k_0})$. Then the R -cost-enriched pre-image of L under f , denoted by $f_R^{-1}(L)$, is a pair $(\mathcal{R}, \vec{t})$ such that

- $\mathcal{R} \subseteq (\Sigma^* \times \mathbb{Z}^{k_1+k_0}) \times \cdots \times (\Sigma^* \times \mathbb{Z}^{k_l+k_0})$;
- $\vec{t} = (t_1, \dots, t_{k_0})$ is a vector of linear integer terms where for each $i \in [k_0]$, t_i is a term whose variables are from $\{r_i^{(1)}, \dots, r_i^{(l)}\}$ which are fresh cost registers and are disjoint from R in $\mathcal{A}$;
- L is equal to the language comprising the k_0 -cost-enriched strings

$$(w_0, t_1 [d_1^{(1)}/r_1^{(1)}, \dots, d_1^{(l)}/r_1^{(l)}], \dots, t_{k_0} [d_{k_0}^{(1)}/r_{k_0}^{(1)}, \dots, d_{k_0}^{(l)}/r_{k_0}^{(l)}]),$$

such that

$$w_0 = f((w_1, \vec{c}_1), \dots, (w_l, \vec{c}_l)) \text{ for some } ((w_1, (\vec{c}_1, \vec{d}_1)), \dots, (w_l, (\vec{c}_l, \vec{d}_l))) \in \mathcal{R},$$

where $\vec{c}_j \in \mathbb{Z}^{k_j}$, $\vec{d}_j = (d_1^{(j)}, \dots, d_{k_0}^{(j)}) \in \mathbb{Z}^{k_0}$ for $j \in [l]$.

The R -cost-enriched pre-image of L under f , say $f_R^{-1}(L) = (\mathcal{R}, \vec{t})$, is said to be CERR-definable if $\mathcal{R}$ is a CERR.

Definition ?? is essentially a semantic definition of the pre-images. For the decision procedure, one desires an effective representation of a CERR-definable $f_R^{-1}(L) = (\mathcal{R}, \vec{t})$ in terms of CEFAs. Namely, a CEFA representation of $(\mathcal{R}, \vec{t})$ (where t_j is over $\{r_j^{(1)}, \dots, r_j^{(l)}\}$ for $j \in [k_0]$) is a tuple $((\mathcal{A}_{i,1}, \dots, \mathcal{A}_{i,l})_{i \in [n]}, \vec{t})$ such that $(\mathcal{A}_{i,1}, \dots, \mathcal{A}_{i,l})_{i \in [n]}$ is a CEFA representation of $\mathcal{R}$, where $R(\mathcal{A}_{i,j}) = (r'_{j,1}, \dots, r'_{j,k_j}, r_1^{(j)}, \dots, r_{k_0}^{(j)})$ for each $i \in [n]$ and $j \in [l]$. (The cost registers $r'_{1,1}, \dots, r'_{1,k_1}, \dots, r'_{l,1}, \dots, r'_{l,k_l}$ are mutually distinct and freshly introduced.)

Example 2 ($\text{substring}_R^{-1}(L)$). Let $\Sigma = \{a\}$ and $L = \{(w, |w|) \mid w \in \mathcal{L}((aa)^*)\}$. Evidently L is a CERL defined by a CEFA $\mathcal{A} = (Q, \Sigma, R, \delta, \{q_0\}, \{q_0\})$ with $Q = \{q_0, q_1\}$, $R = (r_1)$ and $\delta = \{(q_0, a, q_1), (q_1, a, q_0)\}$. Since substring is from $\Sigma^* \times \mathbb{Z}^2$ to Σ^* , $\text{substring}_R^{-1}(L)$, the R -cost-enriched pre-image of L under substring , is the pair $(\mathcal{R}, t)$, where $t = r_1^{(1)}$ (note that in this case $l = 1$, $k_0 = 1$, and $k_1 = 2$) and

$$\mathcal{R} = \{(w, n_1, n_2, n_2) \mid w \in \mathcal{L}(a^*), n_1 \geq 0, n_2 \geq 0, n_1 + n_2 \leq |w|, n_2 \text{ is even}\},$$

which is represented by $(\mathcal{A}', t)$ such that $\mathcal{A}' = (Q', \Sigma, R', \delta', I', F')$, where

- $Q' = Q \times \{p_0, p_1, p_2\}$, (Intuitively, p_0 , p_1 , and p_2 denote that the current position is before the starting position, between the starting position and ending position, and after the ending position of the substring respectively.)

- $R' = (r'_{1,1}, r'_{1,2}, r_1^{(1)})$,
- $I' = \{(q_0, p_0)\}$, $F' = \{(q_0, p_2), (q_0, p_0)\}$ (where (q_0, p_0) is used to accept the 3-cost-enriched strings $(w, n_1, 0, 0)$ with $0 \leq n_1 \leq |w|$), and
- δ' is

$$\left\{ \begin{array}{l} (q_0, p_0) \xrightarrow{a, \eta_1} (q_0, p_0), (q_0, p_0) \xrightarrow{a, \eta_2} (q_1, p_1), (q_1, p_1) \xrightarrow{a, \eta_2} (q_0, p_1), \\ (q_0, p_1) \xrightarrow{a, \eta_2} (q_1, p_1), (q_1, p_1) \xrightarrow{a, \eta_2} (q_0, p_2), (q_0, p_2) \xrightarrow{a, \eta_3} (q_0, p_2) \end{array} \right\},$$

where $\eta_1(r'_{1,1}) = 1$, $\eta_1(r'_{1,2}) = 0$, $\eta_1(r_1^{(1)}) = 0$, $\eta_2(r'_{1,1}) = 0$, $\eta_2(r'_{1,2}) = 1$, and $\eta_2(r_1^{(1)}) = 1$, $\eta_3(r'_{1,1}) = 0$, $\eta_3(r'_{1,2}) = 0$, and $\eta_3(r_1^{(1)}) = 0$.

Therefore, $\text{substring}_R^{-1}(L)$ is CERR-definable.

It turns out that for each string function f in the assignment statements of SL_{int} , the cost-enriched pre-images of CERLs under f are CERR-definable.

Proposition 1. Let L be a CERL defined by a CEFA $\mathcal{A} = (Q, \Sigma, R, \delta, I, F)$. Then for each string function f ranging over $\cdot$, $\text{replaceAll}_{e,u}$, reverse , FFTs $\mathcal{T}$, and substring , $f_R^{-1}(L)$ is CERR-definable. In addition,

- a CEFA representation of $\cdot_R^{-1}(L)$ can be computed in time $\mathcal{O}(|\mathcal{A}|^2)$,
- a CEFA representation of $\text{reverse}_R^{-1}(L)$ (resp. $\text{substring}_R^{-1}(L)$) can be computed in time $\mathcal{O}(|\mathcal{A}|)$,
- a CEFA representation of $(\mathcal{T}(\mathcal{T}))_R^{-1}(L)$ can be computed in time polynomial in $|\mathcal{A}|$ and exponential in $|\mathcal{T}|$,
- a CEFA representation of $(\text{replaceAll}_{e,u})_R^{-1}(L)$ can be computed in time polynomial in $|\mathcal{A}|$ and exponential in $|e|$ and $|u|$.

The proof of Proposition ?? is given in Appendix ??.

4.3 The Decision Procedure

Let S be an SL_{int} program. Without loss of generality, we assume that for every occurrence of assignments of the form $y := \text{substring}(x, t_1, t_2)$, it holds that t_1 and t_2 are integer variables. This is not really a restriction, since, for instance, if in $y := \text{substring}(x, t_1, t_2)$, neither t_1 nor t_2 is an integer variable, then we introduce fresh integer variables i and j , replace t_1, t_2 by i, j respectively, and add $\text{assert}(i = t_1); \text{assert}(j = t_2)$ in S . We present a decision procedure for the path feasibility problem of S which is divided into five steps.

Step I: Reducing to atomic assertions.

Note first that in our language, each assertion is a positive Boolean combination of atomic formulas of the form $x \in \mathcal{A}$ or $t_1 \circ t_2$ (cf. Section ??). Non-deterministically choose, for each assertion $\text{assert}(\varphi)$ of S , a set of atomic formulas $\Phi_\varphi = \{\alpha_1, \dots, \alpha_n\}$ such that φ holds when atomic formulas in Φ_φ are true.

Then each assertion $\text{assert}(\varphi)$ in S with $\Phi_\varphi = \{\alpha_1, \dots, \alpha_n\}$ is replaced by $\text{assert}(\alpha_1); \dots; \text{assert}(\alpha_n)$, and thus S constrains atomic assertions only.

Step II: Dealing with the case splits in the semantics of indexOf_v and substring .

For each integer term of the form $\text{indexOf}_v(x, i)$ in S , nondeterministically choose one of the following five options (which correspond to the semantics of indexOf_v in Section ??).

- (1) Add $\text{assert}(i < 0)$ to S , and replace $\text{indexOf}_v(x, i)$ with $\text{indexOf}_v(x, 0)$ in S .
- (2) Add $\text{assert}(i < 0)$; $\text{assert}(x \in \mathcal{A}_{\Sigma^* \setminus \Sigma^*})$ to S ; replace $\text{indexOf}_v(x, i)$ with -1 in S .
- (3) Add $\text{assert}(i \geq \text{length}(x))$ to S , and replace $\text{indexOf}_v(x, i)$ with -1 in S .
- (4) Add $\text{assert}(i \geq 0)$; $\text{assert}(i < \text{length}(x))$ to S .
- (5) Add

$$\begin{aligned} & \text{assert}(i \geq 0); \text{assert}(i < \text{length}(x)); \text{assert}(j = \text{length}(x) - i); \\ & y := \text{substring}(x, i, j); \text{assert}(y \in \mathcal{A}_{\Sigma^* \setminus \Sigma^*}) \end{aligned}$$

to S , where y is a fresh string variable, j is a fresh integer variable, and $\mathcal{A}_{\Sigma^* \setminus \Sigma^*}$ is an NFA defining the language $\{w \in \Sigma^* \mid v \text{ does not occur as a substring in } w\}$. Replace $\text{indexOf}_v(x, i)$ with -1 in S .

For each assignment $y := \text{substring}(x, i, j)$ in S , nondeterministically choose one of the following three options (which correspond to the semantics of substring in Section ??).

- (1) Add the statements $\text{assert}(i \geq 0)$; $\text{assert}(i + j \leq \text{length}(x))$ to S .
- (2) Add the statements $\text{assert}(i \geq 0)$; $\text{assert}(i \leq \text{length}(x))$; $\text{assert}(i + j > \text{length}(x))$; $\text{assert}(i' = \text{length}(x) - i)$ to S , and replace $y := \text{substring}(x, i, j)$ with $y := \text{substring}(x, i, i')$, where i' is a fresh integer variable.
- (3) Add the statement $\text{assert}(i < 0)$; $\text{assert}(y \in \mathcal{A}_\varepsilon)$ to S , and remove $y := \text{substring}(x, i, j)$ from S , where $\mathcal{A}_\varepsilon$ is the NFA defining the language $\{\varepsilon\}$.

Step III: Removing length and indexOf .

For each term $\text{length}(x)$ in S , we introduce a fresh integer variable i , replace every occurrence of $\text{length}(x)$ by i , and add the statement $\text{assert}(x \in \mathcal{A}_{\text{len}}[i/r_1])$ to S . (See Example ?? for the definition of $\mathcal{A}_{\text{len}}$.)

For each term $\text{indexOf}_v(x, i)$ occurring in S , introduce two fresh integer variables i_1 and i_2 , replace every occurrence of $\text{indexOf}_v(x, i)$ by i_2 , and add the statements $\text{assert}(i = i_1)$; $\text{assert}(x \in \mathcal{A}_{\text{indexOf}_v}[i_1/r_1, i_2/r_2])$ to S .

Step IV: Removing the assignment statements backwards.

Repeat the following procedure until S contains no assignment statements.

Suppose $y := f(x_1, \vec{i}_1, \dots, x_l, \vec{i}_l)$ is the last assignment of S , where $f : (\Sigma^* \times \mathbb{Z}^{k_1}) \times \dots \times (\Sigma^* \times \mathbb{Z}^{k_l}) \rightarrow \Sigma^*$ is a string function and $\vec{i}_j = (i_{j,1}, \dots, i_{j,k_j})$ for each $j \in [l]$.

Let $\{\mathcal{A}_1, \dots, \mathcal{A}_s\}$ be the set of all CEFA's such that $\text{assert}(y \in \mathcal{A}_j)$ occurs in S for every $j \in [s]$. Let $j \in [s]$ and $R(\mathcal{A}_j) = (r_{j,1}, \dots, r_{j,\ell_j})$. Then from Proposition ??, a CEFA representation of $f_{R(\mathcal{A}_j)}^{-1}(\mathcal{L}(\mathcal{A}_j))$,

say $\left(\left(\mathcal{B}_{j,j'}^{(1)}, \dots, \mathcal{B}_{j,j'}^{(l)}\right)_{j' \in [m_j]}, \vec{t}\right)$, can be effectively computed from $\mathcal{A}$ and f , where we write

$$R\left(\mathcal{B}_{j,j'}^{(j'')}\right) = \left((r')_j^{(j'',1)}, \dots, (r')_j^{(j'',k_{j''})}, r_{j,1}^{(j'')}, \dots, r_{j,\ell_j}^{(j'')}\right)$$

for each $j' \in [m_j]$ and $j'' \in [l]$, and $\vec{t} = (t_1, \dots, t_{\ell_j})$. Note that the cost registers $(r')_j^{(1,1)}, \dots, (r')_j^{(1,k_1)}, \dots, (r')_j^{(l,1)}, \dots, (r')_j^{(l,k_l)}, r_{j,1}^{(1)}, \dots, r_{j,\ell_j}^{(1)}, \dots, r_{j,1}^{(l)}, \dots, r_{j,\ell_j}^{(l)}$ are mutually distinct and freshly introduced, moreover, $R\left(\mathcal{B}_{j,j_1'}^{(j'')}\right) = R\left(\mathcal{B}_{j,j_2'}^{(j'')}\right)$ for distinct $j_1', j_2' \in [m_j]$.

Remove $y := f(x_1, \vec{i}_1, \dots, x_l, \vec{i}_l)$, as well as all the statements $\text{assert}(y \in \mathcal{A}_1), \dots, \text{assert}(y \in \mathcal{A}_s)$ from S . For every $j \in [s]$, nondeterministically choose $j' \in [m_j]$, and add the following statements to S ,

$$\text{assert}\left(x_1 \in \mathcal{B}_{j,j'}^{(1)}\right); \dots; \text{assert}\left(x_l \in \mathcal{B}_{j,j'}^{(l)}\right); S_{j,j',\vec{i}_1,\dots,\vec{i}_l}; S_{j,\vec{t}}$$

where

$$\begin{aligned} S_{j,j',\vec{i}_1,\dots,\vec{i}_l} &\equiv \text{assert}\left(i_{1,1} = (r')_{j,j'}^{(1,1)}\right); \dots; \text{assert}\left(i_{1,k_1} = (r')_{j,j'}^{(1,k_1)}\right); \\ &\quad \dots \\ &\quad \text{assert}\left(i_{l,1} = (r')_{j,j'}^{(l,1)}\right); \dots; \text{assert}\left(i_{l,k_l} = (r')_{j,j'}^{(l,k_l)}\right) \end{aligned}$$

and

$$S_{j,\vec{t}} \equiv \text{assert}\left(r_{j,1} = t_1\right); \dots, \text{assert}\left(r_{j,\ell_j} = t_{\ell_j}\right).$$

Step V: Final satisfiability checking.

In this step, S contains no assignment statements and only assertions of the form $\text{assert}(x \in \mathcal{A})$ and $\text{assert}(t_1 \circ t_2)$ where $\mathcal{A}$ are CEFAs and t_1, t_2 are linear integer terms. Let X denote the set of string variables occurring in S . For each $x \in X$, let $\Lambda_x = \{\mathcal{A}_x^1, \dots, \mathcal{A}_x^{s_x}\}$ denote the set of CEFAs $\mathcal{A}$ such that $\text{assert}(x \in \mathcal{A})$ appears in S . Moreover, let ϕ denote the conjunction of all the LIA formulas $t_1 \circ t_2$ occurring in S . It is straightforward to observe that ϕ is over $R' = \bigcup_{x \in X, j \in [s_x]} R(\mathcal{A}_x^j)$. Then the path feasibility of S is reduced to the satisfiability problem of LIA formulas w.r.t. CEFAs (abbreviated as $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem) which is defined as

deciding whether ϕ is satisfiable w.r.t. $(\Lambda_x)_{x \in X}$, namely, whether there are an assignment function $\theta : R' \rightarrow \mathbb{Z}$ and strings $(w_x)_{x \in X}$ such that $\phi[\theta(R')/R']$ holds and $(w_x, \theta(R(\mathcal{A}_x^j))) \in \mathcal{L}(\mathcal{A}_x^j)$ for every $x \in X$ and $j \in [s_x]$.

This $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem is decidable and pspace-complete; The proof can be found in Appendix ??.

Proposition 2. $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ is pspace-complete.

An example for illustrating the decision procedure can be found in Appendix ??.

Complexity analysis of the decision procedure. Step I and Step II can be done in nondeterministic linear time. Step III can be done in linear time. In Step IV, for each input string variable x in S , at most exponentially many CEFAs can be generated for x , each of which is of at most exponential size. Therefore, Step IV can be done in nondeterministic exponential space. By Proposition ??, Step V can be done in exponential space. Therefore, we conclude that the path feasibility problem of SL_{int} programs is in nexpspace, thus in expspace by Savitch’s theorem [?].

Remark 2. In this paper, we focus on functional finite transducers (cf. Section ??). Our decision procedure is applicable to general finite transducers as well with minor adaptation. However, the expspace complexity upper-bound does not hold any more, because the distributive property $f^{-1}(L_1 \cap L_2) = f^{-1}(L_1) \cap f^{-1}(L_2)$ for regular languages L_1, L_2 only holds for functional finite transducers f .

5 Evaluations

We have implemented the decision procedure presented in the preceding section based on the recent string constraint solver OSTRICH [?], resulting in a new solver OSTRICH+. OSTRICH is written in Scala and based on the SMT solver Princess [?]. OSTRICH+ reuses the parser of Princess, but replaces the NFAs from OSTRICH with CEFAs. Correspondingly, in OSTRICH+, the pre-image computation for concatenation, **replaceAll**, **reverse**, and finite transducers is reimplemented, and a new pre-image operator for **substring** is added. OSTRICH+ also implements CEFA constructions for **length** and **indexOf**. More details can be found in Appendix ??.

We have compared OSTRICH+ with some of the state-of-the-art solvers on a wide range of benchmarks. We discuss the benchmarks in Section ?? and present the experimental results in Section ??.

5.1 Benchmarks

Our evaluation focuses on problems that combine string with integer constraints. To this end, we consider the following four sets of benchmarks, all in SMT-LIB 2 format.

Transducer+ is derived from the Transducer benchmark suite of OSTRICH [?]. The Transducer suite involves seven transducers: **toUpper** (replacing all lower-case letters with their uppercase ones) and its dual **toLower**, **htmlEscape** and its dual **htmlUnescape**, **escapeString**, **addslashes**, and **trim**. These transducers are collected from Stranger [?] and SLOTH [?]. Initially none of the benchmarks involved integers. In Transducer+, we encode four security-relevant properties of transducers [?], with the help of the functions **charAt** and **length**:

- idempotence: given $\mathcal{T}$, whether $\forall x. \mathcal{T}(\mathcal{T}(x)) = \mathcal{T}(x)$;

- duality: given $\mathcal{T}_1$ and $\mathcal{T}_2$, whether $\forall x. \mathcal{T}_2(\mathcal{T}_1(x)) = x$;
- commutativity: given $\mathcal{T}_1$ and $\mathcal{T}_2$, whether $\forall x. \mathcal{T}_2(\mathcal{T}_1(x)) = \mathcal{T}_1(\mathcal{T}_2(x))$;
- equivalence: given $\mathcal{T}_1$ and $\mathcal{T}_2$, whether $\forall x. \mathcal{T}_1(x) = \mathcal{T}_2(x)$.

For instance, we encode the non-idempotence of $\mathcal{T}$ into the path feasibility of the SL_{int} program $y := \mathcal{T}(x); z := \mathcal{T}(y); S_{y \neq z}$, where y and z are two fresh string variables, and $S_{y \neq z}$ is the SL_{int} program encoding $y \neq z$ (see Appendix ?? for the details). We also include in Transducer+ three instances generated from a program to sanitize URLs against XSS attacks (see Appendix ?? for the details), where $\mathcal{T}_{\text{trim}}$ is used. In total, we obtain 94 instances for the Transducer+ suite.

SLOG+ is adapted from the SLOG benchmark suite [?], containing 3,511 instances about strings only. We obtain SLOG+ by choosing a string variable x for each instance, and adding the statement `assert (length( $x$ ) < 2 indexOf $a$ ( $x$ , 0))` for some $a \in \Sigma$. As in [?], we split SLOG+ into SLOG+(replace) and SLOG+(replaceall), comprising 3,391 and 120 instances respectively. In addition to the `indexOf` and `length` functions, the benchmarks use regular constraints and concatenation; SLOG+(replace) also contains the `replace` function (replacing the first occurrence), while SLOG+(replaceall) uses the `replaceAll` function (replacing all occurrences).

PyEx [?] contains 25,421 instances derived by the PyEx tool, a symbolic execution engine for Python programs. The PyEx suite was generated by the CVC4 group from four popular Python packages: `httplib2`, `pip`, `pymongo`, and `requests`. These instances use regular constraints, concatenation, `length`, `substring`, and `indexOf` functions. Following [?], the PyEx suite is further divided into three parts: PyEx-td, PyEx-z3 and PyEx-zz, comprising 5,569, 8,414 and 11,438 instances, respectively.

Kaluza [?] is the most well-known benchmark suite in literature, containing 47,284 instances with regular constraints, concatenation, and the `length` function. The 47,284 benchmarks include 28,032 satisfiable and 9,058 unsatisfiable problems in SSA form.

5.2 Experiments

We compare OSTRICH+ to CVC4 [?], Z3-str3 [?], and Z3-Trau [?], as well as two configurations of OSTRICH [?] with standard NFAs. The configuration OSTRICH⁽¹⁾ is a direct implementation of the algorithm in [?], and does not support integer functions. In OSTRICH⁽²⁾, we integrated support for the `length` function as in Norn [?], based on the computation of length abstractions of regular languages, and handle `indexOf`, `substring`, and `charAt` via an encoding to word equations. The experiments are executed on a computer with an Intel Xeon Silver 4210 2.20GHz and 2.19GHz CPU (2-core) and 8GB main memory, running 64bit Ubuntu 18.04 LTS OS and Java 1.8. We use a timeout of 30 seconds (wall-clock time), and report the number of satisfiable and unsatisfiable problems solved by each of the systems. Table ?? summarises the experimental results. We did not observe incorrect answers by any tool.

Benchmark	Output	CVC4	Z3-str3	Z3-Trau	OSTRICH ⁽¹⁾	OSTRICH ⁽²⁾	OSTRICH+
Transducer+ Total: 94	sat	–	–	–	0	0	84
	unsat	–	–	–	1	1	4
	inconcl.	–	–	–	93	93	6
SLOG+(replaceall) Total: 120	sat	104	–	–	0	0	98
	unsat	11	–	–	7	5	12
	inconcl.	5	–	–	113	115	10
SLOG+(replace) Total: 3,391	sat	1,309	878	–	0	169	584
	unsat	2,082	2,066	–	2,079	2,075	2,082
	inconcl.	0	447	–	1,312	1,147	725
PyEx-td Total: 5,569	sat	4,224	4,068	4,266	68	96	4,141
	unsat	1,284	1,289	1,295	95	93	1,203
	inconcl.	61	212	8	5,406	5,380	225
PyEx-z3 Total: 8,414	sat	6,346	6,040	7,003	76	100	5,489
	unsat	1,358	1,370	1,394	61	53	1,239
	inconcl.	710	1,004	17	8,277	8,261	1,686
PyEx-zz Total: 11,438	sat	10,078	8,804	10,129	71	98	9,033
	unsat	1,204	1,207	1,222	91	61	868
	inconcl.	156	1,427	87	11,276	11,279	1,537
Kaluza Total: 47,284	sat	35,264	33,438	34,769	23,397	28,522	27,962
	unsat	12,014	11,799	12,014	10,445	10,445	9,058
	inconcl.	6	2,047	501	13,442	8,317	10,264
Total: 76,310	solved	75,278	70,959	72,092	36,391	41,718	61,857
	unsolved	1,032	5,351	4,218	39,919	34,592	14,453

Table 1. Experimental results on different benchmark suites. ‘–’ means that the tool is not applicable to the benchmark suite, and ‘inconclusive’ means that a tool gave up, timed out, or crashed.

There are two additional state-of-the-art solvers Slent and Trau+ which were not included in the evaluation. We exclude Slent [?] because it uses its own input format laut, which is different from the SMT-LIB 2 format used for our benchmarks; also, Transducer+ is beyond the scope of Slent. Trau+ [?] integrates Trau with Sloth to deal with both finite transducers and integer constraints. We were unfortunately unable to obtain a working version of Trau+, possibly because Trau requires two separate versions of Z3 to run. In addition, the algorithm in [?] focuses on length-preserving transducers, which means that Transducer+ is beyond the scope of Trau+.

OSTRICH+ and OSTRICH are the only tools applicable to the problems in Transducer+. With a timeout of 30s, OSTRICH+ can solve 88 of the benchmarks, but this number rises to 94 when using a longer timeout of 600s. Given the complexity of those benchmarks, this is an encouraging result. OSTRICH can only solve one of the benchmarks, because the encoding of `charAt` in the benchmarks using equations almost always leads to problems that are not in SSA form.

On SLOG+(replaceall), OSTRICH+ and CVC4 are very close: OSTRICH+ solves 98 satisfiable instances, slightly less than the 104 instances solved by CVC4, while OSTRICH+ solves one more unsatisfiable instance than CVC4 (12 versus 11). The suite is beyond the scope of Z3-str3 and Z3-Trau, which do not support `replaceAll`.

On SLOG+(replace), OSTRICH+, CVC4, and Z3-str3 solve a similar number of unsatisfiable problems, while CVC4 solves the largest number of satisfiable instances (1,309). The suite is beyond the scope of Z3-Trau which does not support `replace`.

On the three PyEx suites, Z3-Trau consistently solves the largest number of instances by some margin. OSTRICH+ solves a similar number of instances as Z3-str3. Interpreting the results, however, it has to be taken into account that PyEx includes 1,334 instances that are not in SSA form, which are beyond the scope of OSTRICH+.

The Kaluza problems can be solved most effectively by CVC4. OSTRICH+ can solve almost all of the around 80% of the benchmarks that are in SSA form, however.

OSTRICH+ consistently outperforms OSTRICH⁽¹⁾ and OSTRICH⁽²⁾ in the evaluation, except for the Kaluza benchmarks. For OSTRICH⁽¹⁾, this is expected because most benchmarks considered here contain integer functions. For OSTRICH⁽²⁾, it turns out that the encoding of `indexOf`, `substring`, and `charAt` as word equations usually leads to problems that are not in SSA form, and therefore are beyond the scope of OSTRICH.

In summary, we observe that OSTRICH+ is competitive with other solvers, while is able to handle benchmarks that are beyond the scope of the other tools due to the combination of string functions (in particular transducers) and integer constraints. Interestingly, the experiments show that OSTRICH+, at least in its current state, is better at solving unsatisfiable problems than satisfiable problems; this might be an artefact of the use of nuXmv for analysing products of CEFAs. We expect that further optimisation of our algorithm will lead to additional performance improvements. For instance, a natural optimisation that is to be included in our implementation is to use standard finite automata, as opposed to CEFAs, for simpler problems such as the Kaluza benchmarks. Such a combination of automata representations is mostly an engineering effort.

6 Conclusion

In this paper, we have proposed an expressive string constraint language which can specify constraints on both strings and integers. We provided an automata-theoretic decision procedure for the path feasibility problem of this language. The decision procedure is simple, generic, and amenable to implementation, giving rise to a new solver OSTRICH+. We have evaluated OSTRICH+ on a wide range of existing and newly created benchmarks, and have obtained very encouraging results. OSTRICH+ is shown to be the first solver which is capable of tackling finite transducers and integer constraints with completeness

guarantees. Meanwhile, it demonstrates competitive performance against some of the best state-of-the-art string constraint solvers.

A The $\mathbf{SL}_{\text{int}}$ program $S_{x \neq y}$ encoding $x \neq y$

At first, we note that the function $\text{charAt}(x, i)$ which returns $x[i]$ (i.e., the character of x at the position i) can be seen as a special case of **substring**, namely $\text{charAt}(x, i) \equiv \text{substring}(x, i, 1)$. Then the string inequality $x \neq y$ is expressed as the following $\mathbf{SL}_{\text{int}}$ program (denoted by $S_{x \neq y}$)

$$\begin{aligned} z_1 &:= \text{charAt}(x, i); z_2 := \text{charAt}(y, i); \\ \text{assert } (\text{length}(x) \neq \text{length}(y) \vee \bigvee_{a \in \Sigma} (z_1 \in \mathcal{A}_a \wedge z_2 \in \mathcal{A}_{\Sigma \setminus \{a\}})), \end{aligned}$$

where z_1, z_2 are two freshly introduced string variables, and $\mathcal{A}_a$ (resp. $\mathcal{A}_{\Sigma \setminus \{a\}}$) is the NFA accepting $\{a\}$ (resp. $\Sigma \setminus \{a\}$). Intuitively, two strings are different if their lengths are different or otherwise, there exists some position where the characters of the two strings are different.

B Construction of $\mathcal{A}_{\text{indexOf}_v}$

In this section, we show that the function $\text{indexOf}_v(\cdot, \cdot)$ can be captured by CEFA. We start with the simple example for $v = a$.

Example 3 (CEFA for indexOf_a). Let $a \in \Sigma$. Then $\mathcal{A}_{\text{indexOf}_a} = ((q_0, q_1, q_2), \Sigma, (r_1, r_2), \delta_{\text{indexOf}_a}, \{q_0\}, \{q_2\})$, where $\delta_{\text{indexOf}_a}$ comprises the tuples

- (q_0, b, q_0, η) such that $b \in \Sigma, \eta(r_1) = 1, \eta(r_2) = 1$,
- (q_0, b, q_1, η) such that $b \in \Sigma, \eta(r_1) = 0, \eta(r_2) = 1$,
- (q_0, a, q_2, η) such that $\eta(r_1) = 0, \eta(r_2) = 0$,
- (q_1, b, q_1, η) such that $b \in \Sigma \setminus \{a\}, \eta(r_1) = 0, \eta(r_2) = 1$,
- (q_1, a, q_2, η) such that $\eta(r_1) = 0, \eta(r_2) = 0$,
- (q_2, b, q_2, η) such that $b \in \Sigma, \eta(r_1) = 0, \eta(r_2) = 0$.

Intuitively, r_1 corresponds to the starting position i of $\text{indexOf}_a(x, i)$, r_2 corresponds to the output of $\text{indexOf}_a(x, i)$, q_0 specifies that the current position is before i , q_1 specifies that the current position is after i , while a has not occurred yet, and q_2 specifies that a has occurred after i .

Technically, for any NFA $\mathcal{A}$ and constant string v , we can construct a CEFA accepting $\{(w, (n, \text{indexOf}_v(w, n))) \mid w \in \mathcal{L}(\mathcal{A}), n \leq \text{indexOf}_v(w, n) < |w|\}$. For this purpose, we need a concept of window profiles of string positions w.r.t. v , which are elements of $\{\perp, \top\}^{n-1}$. The window profiles facilitate recognising the first occurrence of v in the input string. Intuitively, given a string u , the window profile of a position i in u w.r.t. v encodes the matchings of prefixes of v to the suffixes of $u[0, i]$ (see [?] for the details). For $\pi = \pi_1 \cdots \pi_{n-1} \in \{\perp, \top\}^{n-1}$ and $b \in \Sigma$, we use $\text{uwp}(\vec{\pi}, b)$ to represent the window profile updated from π after reading the letter b , specifically, $\text{uwp}(\vec{\pi}, b) = \vec{\pi}'$ such that

- $\pi'_1 = \top$ iff $b = a_1$,
- for each $i \in [n-2]$, $\pi'_{i+1} = \top$ iff $\pi_i = \top$ and $b = a_{i+1}$.

Let WP_v denote the set of window profiles of string positions w.r.t. v . From the result in [?], we know that $|WP_v| \leq |v|$.

Suppose $v = a_1 \cdots a_n$ with $n \geq 2$. Then indexOf_v is captured by the CEFA $\mathcal{A}_{\text{indexOf}_v} = (Q, \Sigma, R, \delta, I, F)$, such that

- $Q = \{q_0, q_1\} \cup WP_v \cup WP_v \times [n]$,
- $R = (r_1, r_2)$ (where r_1, r_2 represent the input and output positions of indexOf_v respectively),
- $I = \{q_0\}$,
- $F = \{q_1\}$, and
- δ comprises
 - the tuples (q_0, a, q_0, η) such that $a \in \Sigma$, $\eta(r_1) = 1$, and $\eta(r_2) = 1$,
 - the tuples $(q_0, a, \vec{\pi}, \eta)$ such that $a \in \Sigma$, $\vec{\pi} = \theta \perp^{n-2}$ where $\theta = \top$ iff $a = a_1$, $\eta(r_1) = 0$, and $\eta(r_2) = 0$ (recall that the first position of a string is 0),
 - the tuples $(\vec{\pi}, a, \text{uwp}(\vec{\pi}, a), \eta)$ such that $\vec{\pi} \in WP_u$, $a \in \Sigma$, $\pi_{n-1} = \perp$ or $a \neq a_n$, $\eta(r_1) = 0$, and $\eta(r_2) = 1$,
 - the tuples $(\vec{\pi}, a, (\text{uwp}(\vec{\pi}, a), 1), \eta)$ such that $\vec{\pi} \in WP_u$, $a = a_1$, $\pi_{n-1} = \perp$ or $a \neq a_n$, $\eta(r_1) = 0$, and $\eta(r_2) = 1$,
 - the tuples $((\vec{\pi}, i), a, (\text{uwp}(\vec{\pi}, a), i+1), \eta)$ such that $\vec{\pi} \in WP_u$, $i \in [n-2]$, $a = a_{i+1}$, $\pi_{n-1} = \perp$ or $a \neq a_n$, $\eta(r_1) = 0$, and $\eta(r_2) = 0$,
 - the tuples $((\vec{\pi}, n-1), a, q_1, \eta)$ such that $\vec{\pi} \in WP_u$, $a = a_n$, $\eta(r_1) = 0$, and $\eta(r_2) = 0$,
 - the tuples (q_1, a, q_1, η) such that $a \in \Sigma$, $\eta(r_1) = 0$, and $\eta(r_2) = 0$.

C Proof of Proposition ??

Proposition ??. Let L be a CERL defined by a CEFA $\mathcal{A} = (Q, \Sigma, R, \delta, I, F)$. Then for each string function f ranging over $\cdot$, **replaceAll** _{e, u} , **reverse**, FFTs $\mathcal{T}$, and **substring**, $f_R^{-1}(L)$ is CERR-definable. In addition,

- a CEFA representation of $\cdot_R^{-1}(L)$ can be computed in time $O(|\mathcal{A}|^2)$,
- a CEFA representation of **reverse** _{R} ⁻¹(L) (resp. **substring** _{R} ⁻¹(L)) can be computed in time $O(|\mathcal{A}|)$,
- a CEFA representation of $(\mathcal{T}(\mathcal{T}))_R^{-1}(L)$ can be computed in time polynomial in $|\mathcal{A}|$ and exponential in $|\mathcal{T}|$,
- a CEFA representation of **replaceAll** _{e, u} ⁻¹(L) can be computed in time polynomial in $|\mathcal{A}|$ and exponential in $|e|$ and $|u|$.

Proof. Let $\mathcal{A} = (Q, \Sigma, R, \delta, I, F)$ be a CEFA with $R = (r_1, \dots, r_k)$. We show how to construct a CEFA representation of $f_R^{-1}(L)$ for each function f in SL_{int} .

$\cdot_R^{-1}(L)$. A CEFA representation of $\cdot_R^{-1}(L)$ is given by $((\mathcal{A}_{I,q}, \mathcal{A}_{q,F})_{q \in Q}, \vec{t})$, where

- $\mathcal{A}_{I,q} = (Q, \Sigma, R^{(1)}, \delta^{(1)}, I, \{q\})$ and $\mathcal{A}_{q,F} = (Q, \Sigma, R^{(2)}, \delta^{(2)}, \{q\}, F)$ such that
 - $R^{(1)} = (r_1^{(1)}, \dots, r_k^{(1)})$, $R^{(2)} = (r_1^{(2)}, \dots, r_k^{(2)})$,
 - $\delta^{(1)}$ comprises the tuples (q, a, q', η') satisfying that there exists η such that $(q, a, q', \eta) \in \delta$ and for each $j \in [k]$, and $\eta'(r_j^{(1)}) = \eta(r_j)$, similarly for $\delta^{(2)}$,
- and $\vec{t} = (r_1^{(1)} + r_1^{(2)}, \dots, r_k^{(1)} + r_k^{(2)})$.

Note that the size of $((\mathcal{A}_{I,q}, \mathcal{A}_{q,F})_{q \in Q}, \vec{t})$ is $O(|\mathcal{A}|^2)$.

$\text{reverse}_R^{-1}(L)$. A CEFA representation of $\text{reverse}_R^{-1}(L)$ is given by $(\mathcal{A}^{(r)}, \vec{t})$, where

- $\mathcal{A}^{(r)} = (Q, \Sigma, R^{(1)}, \delta', F, I)$ such that
 - $R^{(1)} = (r_1^{(1)}, \dots, r_k^{(1)})$, and
 - δ' comprises the tuples (q', a, q, η') satisfying that there exists η such that $(q, a, q', \eta) \in \delta$, and $\eta'(r_i^{(1)}) = \eta(r_i)$ for each $i \in [k]$,
- and $\vec{t} = (r_1^{(1)}, \dots, r_k^{(1)})$.

Note that $\mathcal{L}(\mathcal{A}^{(r)}) = \{(w^{(r)}, \vec{n}) \mid (w, \vec{n}) \in \mathcal{L}(\mathcal{A})\}$, and the size of $(\mathcal{A}^{(r)}, \vec{t})$ is $O(|\mathcal{A}|)$.

$\text{substring}_R^{-1}(L)$. A CEFA representation of $\text{substring}_R^{-1}(L)$ is given by $(\mathcal{B}, \vec{t})$, where

- $\mathcal{B} = (Q', \Sigma, R', \delta', I', F')$ such that
 - $Q' = Q \times \{p_0, p_1, p_2\}$, (intuitively, p_0 , p_1 , and p_2 denote that the current position is before the starting position, between the starting position and ending position, and after the ending position respectively)
 - $R' = (r'_{1,1}, r'_{1,2}, r'_1, \dots, r'_k)$, (intuitively, $r'_{1,1}$ denotes the starting position, and $r'_{1,2}$ denotes the length of the substring)
 - $I' = I \times \{p_0\}$, $F' = F' \times \{p_2\} \cup (I \cap F) \times \{p_0\}$,
 - and δ' comprises
 - * the tuples $((q, p_0), a, (q, p_0), \eta')$ such that $q \in I$, $a \in \Sigma$, and η' satisfies that $\eta'(r'_{1,1}) = 1$, and $\eta'(r'_{1,2}) = 0$, and $\eta'(r_j^{(1)}) = 0$ for each $j \in [k]$,
 - * the tuples $((q, p_0), a, (q', p_1), \eta')$ such that $q \in I$ and there exists η satisfying that $(q, a, q', \eta) \in \delta$, moreover, $\eta'(r'_{1,1}) = 0$ (recall that the positions of strings start at 0), $\eta'(r'_{1,2}) = 1$, and $\eta'(r_j^{(1)}) = \eta(r_j)$ for each $j \in [k]$,
 - * the tuples $((q, p_0), a, (q', p_2), \eta')$ such that $q \in I$ and there exists η satisfying that $(q, a, q', \eta) \in \delta$, moreover, $q' \in F$, and $\eta'(r'_{1,1}) = 0$ (recall that the positions of strings start at 0), $\eta'(r'_{1,2}) = 1$, and $\eta'(r_j^{(1)}) = \eta(r_j)$ for each $j \in [k]$,
 - * the tuples $((q, p_1), a, (q', p_1), \eta')$ such that there exists η satisfying that $(q, a, q', \eta) \in \delta$, $\eta'(r'_{1,1}) = 0$, and $\eta'(r'_{1,2}) = 1$, and $\eta'(r_j^{(1)}) = \eta(r_j)$ for each $j \in [k]$,
 - * the tuples $((q, p_1), a, (q', p_2), \eta')$ such that $q' \in F$, and there exists η satisfying that $(q, a, q', \eta) \in \delta$, moreover, $\eta'(r'_{1,1}) = 0$, $\eta'(r'_{1,2}) = 1$, and $\eta'(r_j^{(1)}) = \eta(r_j)$ for each $j \in [k]$,
 - * the tuples $((q, p_2), a, (q, p_2), \eta')$ such that $q \in F$, $\eta'(r'_{1,1}) = 0$, and $\eta'(r'_{1,2}) = 0$, and $\eta'(r_j^{(1)}) = 0$ for each $j \in [k]$,
- $\vec{t} = (r_1^{(1)}, \dots, r_k^{(1)})$.

Note that the size of $(\mathcal{B}, \vec{t})$ is $O(|\mathcal{A}|)$.

$(\mathcal{T})_R^{-1}(L)$. Suppose $\mathcal{T} = (Q', \Sigma, \delta', I', F')$. Then a CEFA representation of $(\mathcal{T})_R^{-1}(L)$ is given by $(\mathcal{B}, \vec{t})$, where

- $\mathcal{B}$ simulates the run of $\mathcal{T}$ on the input string, meanwhile, it simulates the run of $\mathcal{A}$ on the output string of $\mathcal{T}$, formally, $\mathcal{B} = (Q' \times Q, \Sigma, R^{(1)}, \delta'', I' \times I, F' \times F)$ such that
 - $R^{(1)} = (r_1^{(1)}, \dots, r_k^{(1)})$, and
 - δ'' comprises the tuples $((q'_1, q_1), a, (q'_2, q_2), \eta')$ satisfying one of the following conditions,
 - * there exist $u = a_1 \cdots a_n \in \Sigma^+$ and a transition sequence $p_0 \xrightarrow[\delta]{a_1, \eta_1} p_2 \cdots p_{n-1} \xrightarrow[\delta]{a_n, \eta_n} p_n$ in $\mathcal{A}$ such that $(q'_1, a, q'_2, u) \in \delta'$, $p_0 = q_1$, $p_n = q_2$, and for each $j \in [k]$, $\eta'(r_j^{(1)}) = \eta_1(r_j) + \cdots + \eta_n(r_j)$,
 - * $(q'_1, a, q'_2, \varepsilon) \in \delta'$, $q_1 = q_2$, and $\eta'(r_j^{(1)}) = 0$ for each $j \in [k]$,
- $\vec{t} = (r_1^{(1)}, \dots, r_k^{(1)})$.

Note that the number of transitions of $\mathcal{B}$ can be exponential in the worst case, since it summarises the updates of cost registers of $\mathcal{A}$ on the output strings of the transitions of $\mathcal{T}$. More precisely, let

- ℓ be the maximum length of the output strings of transitions of $\mathcal{T}$,
- N be the maximum number of transitions between a given pair of states of $\mathcal{A}$, and
- C be the maximum absolute value of the integer constants occurring in $\mathcal{A}$,

then $|\delta'|$, the cardinality of δ'' , is bounded by $|\delta'| \times |Q|^2 \times N^\ell$, and the integer constants occurring in each transition of δ'' are bounded by ℓC . Therefore, the size of $(\mathcal{B}, \vec{t})$ is

$$O(|\delta'| \times |Q|^2 \times N^\ell \times k \log_2(\ell C)).$$

Since $|\delta'|, \ell \leq |\mathcal{T}|$, $|Q|, N, k \leq |\mathcal{A}|$, and $C \leq 2^{|\mathcal{A}|}$, we deduce that the size of $(\mathcal{B}, \vec{t})$ is $O(|\mathcal{T}| \times |\mathcal{A}|^2 \times |\mathcal{A}|^{|\mathcal{T}|} \times |\mathcal{A}|^2 \log_2(|\mathcal{T}|)) = |\mathcal{A}|^{O(|\mathcal{T}|)} |\mathcal{T}| \log_2(|\mathcal{T}|)$.

$(\text{replaceAll}_{e,u})_R^{-1}(L)$. From the result in [?], we know that a NFT $\mathcal{T}_{e,u} = (Q', \Sigma, \delta', I', F')$ can be constructed to capture $\text{replaceAll}_{e,u}$. Moreover,

- $|Q'|$, as well as $|\delta'|$, is $2^{O(|e|)}$,
- ℓ , the maximum length of the output strings of transitions of $\mathcal{T}_{e,u}$, is $|u|$.

Then a CEFA representation of $(\text{replaceAll}_{e,u})_R^{-1}(L)$ can be constructed as that of $(\mathcal{T}_{e,u})_R^{-1}(L)$. Let N denote the maximum number of transitions between a given pair of states of $\mathcal{A}$, and C be the maximum absolute value of the integer constants occurring in $\mathcal{A}$, which is bounded by $2^{|\mathcal{A}|}$. Then the CEFA representation of $(\text{replaceAll}_{e,u})_R^{-1}(L)$ is of size

$$O(|\delta'| \times |Q|^2 \times N^\ell \times k \log_2(\ell C)) = 2^{O(|e|)} |\mathcal{A}|^2 |\mathcal{A}|^{|u|} |\mathcal{A}|^2 \log_2 |u| = 2^{O(|e|)} |\mathcal{A}|^{O(|u|)}.$$

according to the aforementioned discussion for NFTs. $\square$

D Proof of Proposition ??

Proposition ??. The $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem is pspace-complete.

Proof. The lower bound follows from the pspace-hardness of the intersection problem of NFAs.

For the upper bound, let $\{\mathcal{A}_i^j\}_{i \in I, j \in J_i}$ be a family of CEFAs each of which carries a vector of registers R_i^j and ϕ be a quantifier-free LIA formula such that R_i^j are pairwise disjoint and the variables of ϕ are from $R' := \bigcup_{i,j} R_i^j$.

First, we observe that we can focus on monotonic CEFAs where the cost registers are monotone in the sense that their values are non-decreasing during the course of execution. In other words, they can only be updated with natural number (as opposed to general integer) constants. This observation is justified by the following reduction.

For each register $r \in R_j^i$, we introduce two registers r^+, r^- . Let $(R_i^j)^\pm$ denote the vector of registers by replacing each $r \in R_j^i$ with (r^+, r^-) . Intuitively, for each $r \in R_j^i$, the updates of r in $\mathcal{A}_i^j$ are split into non-negative ones and negative ones, with the former stored in r^+ and the latter in r^- . Suppose $(R')^\pm = \bigcup_{i,j} (R_i^j)^\pm$. Then we construct monotonic CEFAs $(\mathcal{B}_i^j)_{i \in I, j \in J_i}$ and an LIA formula $\phi^\pm$ such that

there are an assignment function $\theta : R' \rightarrow \mathbb{Z}$ and strings $(w_i)_{i \in I}$ such that $\phi[\theta(R')/R']$ holds and $(w_i, \theta(R_i^j)) \in \mathcal{L}(\mathcal{A}_i^j)$ for every $i \in I$ and $j \in J_i$

if and only if

there are an assignment function $\theta^\pm : (R')^\pm \rightarrow \mathbb{N}$ and strings $(w_i)_{i \in I}$ such that $\phi^\pm[\theta^\pm((R')^\pm)/(R')^\pm]$ holds and $(w_i, \theta^\pm((R_i^j)^\pm)) \in \mathcal{L}(\mathcal{B}_i^j)$ for every $i \in I$ and $j \in J_i$.

For $i \in I$ and $j \in J_i$, the CEFA $\mathcal{B}_i^j$ is obtained from $\mathcal{A}_i^j$ by replacing each transition (q, a, q', η) in $\mathcal{A}_i^j$ by the transition (q, a, q', η') such that for each $r \in R_j^i$,

$$\eta'(r^+) = \begin{cases} \eta(r), & \text{if } \eta(r) \geq 0 \\ 0 & \text{otherwise} \end{cases}, \eta'(r^-) = \begin{cases} 0, & \text{if } \eta(r) \geq 0 \\ -\eta(r) & \text{otherwise} \end{cases}.$$

In addition, $\phi^\pm$ is obtained from ϕ by replacing each $r \in R'$ with $r^+ - r^-$.

It remains to prove the $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem for monotonic CEFAs is in pspace, namely,

given a family of monotonic CEFAs $\{\mathcal{A}_i^j\}_{i \in I, j \in J_i}$ each of which carries a vector of registers R_i^j and a quantifier-free LIA formula ϕ such that R_i^j are pairwise disjoint, and the variables of ϕ are from $R' = \bigcup_{i,j} R_i^j$, deciding whether there are an assignment function $\theta : R' \rightarrow \mathbb{N}$ and strings $(w_i)_{i \in I}$ such that $\phi[\theta(R')/R']$ holds and $(w_i, \theta(R_i^j)) \in \mathcal{L}(\mathcal{A}_i^j)$ for every $i \in I$ and $j \in J_i$ is in pspace.

We use Proposition 16 in [?] to show the result. Proposition 16 in [?] mainly considered monotonic counter machines, which can be seen as monotonic CEFAs where each transition contains no alphabet symbol, and $\eta(r) \in \{0, 1\}$ for the update function η therein.

For each $i \in I$ and $j \in J_i$, let $(\mathcal{A}')_i^j$ be the monotonic counter machine obtained from $\mathcal{A}_i^j$ by the following two-step procedure:

1. [Remove the alphabet symbols]: Remove alphabet symbols a in each transition (q, a, q', η) of $\mathcal{A}_i^j$.
2. [From binary encoding to unary encoding]: Replace each transition (q, q', η) such that $\ell = \max_{r \in R_i^j} \eta(r) > 1$ with a sequence of transitions $(q, p_1, \eta'_1), \dots, (p_{\ell-1}, q', \eta'_\ell)$, where $p_1, \dots, p_{\ell-1}$ are the freshly introduced states, moreover, $\eta'_j(r) = 1$ if $\eta(r) \geq j$, and $\eta'_j(r) = 0$ otherwise.

According to Proposition 16 in [?], we have the following property.

Given a family of monotonic counter machines $\{C_i\}_{i \in I}$ each of which carries a vector of counters R_i and a quantifier-free LIA formula ϕ such that R_i are pairwise disjoint, and the variables of ϕ are from $R' = \bigcup_i R_i$. If there is an assignment function $\theta : R' \rightarrow \mathbb{N}$ such that $\phi[\theta(R')/R']$ holds and $\theta(R_i)$ is a reachable valuation of counters in C_i for every $i \in I$, then there are desired θ such that for each $i \in I$ and $r \in R_i$, $\theta(r)$ is at most polynomial in the number of states in C_i , exponential in $|R_i|$, and exponential in $|\phi|$.

For each $i \in I$, let C_i be the product of monotonic counter machines $(\mathcal{A}')_i^j$ for $j \in J_i$. From the fact that the number of states of $(\mathcal{A}')_i^j$ is at most the product of the number of transitions of $\mathcal{A}_i^j$ and $B_{\mathcal{A}_i^j}$ (where $B_{\mathcal{A}_i^j}$ denotes the maximum natural number constants $\eta(r)$ in $\mathcal{A}_i^j$), we deduce the following,

if there are an assignment function $\theta : R' \rightarrow \mathbb{N}$ and strings $(w_i)_{i \in I}$ such that $\phi[\theta(R')/R']$ holds and $(w_i, \theta(R_i^j)) \in \mathcal{L}(\mathcal{A}_i^j)$ for every $i \in I$ and $j \in J_i$, then there are desired θ and $(w_i)_{i \in I}$ such that for each $i \in I$ and $r \in \bigcup_{j \in J_i} R_i^j$, $\theta(r)$ is at most polynomial in the product of the number of transitions in $\mathcal{A}_i^j$ and $B_{\mathcal{A}_i^j}$ for $j \in J_i$, exponential in $|\bigcup_{j \in J_i} R_i^j|$, and exponential in $|\phi|$.

Since the values of all the registers in $\mathcal{A}_i^j$ for $i \in I$ and $j \in J_i$ can be assumed to be at most exponential, and thus their binary encodings can be stored in polynomial space, one can nondeterministically guess the strings $(w_i)_{i \in I}$, and for each $i \in I$ and $j \in J_i$, simulate the runs of CEFAs $\mathcal{A}_i^j$ on w_i , and finally evaluate ϕ with the register values after all $\mathcal{A}_i^j$ accept, in polynomial space. From Savitch's theorem [?], we conclude that the $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem for monotonic CEFAs is in pspace. This concludes the proof of the proposition. $\square$

E An example `urlXssSanitise(url)` for sanitising URLs

We use the following JavaScript program as the running example, which defines a function `urlXssSanitise` for sanitising URLs. A typical URL consists of a hierarchical sequence of components commonly referred to as protocol, host, path, query, and fragment. For instance, in “`http://www.example.com/some/abc.html?name=john#print`”, the protocol is “`http`”, the host is “`www.example.com`”, the path is “`/some/abc.html`”, the query is “`name=john`” (preceded by `?`), and the fragment is “`print`” (preceded by `#`). Both the query and the fragment could be empty in a URL. The aim of `urlXssSanitise` is to mitigate URL reflection attacks, by filtering out the dangerous substring “`script`” from the query and fragment components of the input URL. URL reflection attacks are a type of cross-site-scripting (XSS) attacks that do not rely on saving malicious code in database, but rather on hiding it in the query or fragment component of URLs, e.g., “`http://www.example.com/some/abc.html?name=<script>alert('xss!');</script>`”.

<MINTED>

Note that `urlXssSanitise` uses the JavaScript sanitisation operation `trim` that removes whitespace from both ends of a string, which can be conveniently modelled by finite-state transducers. Two string functions involving the integer data type, namely, `indexOf` and `substr` (`indexOf` and `substring` in this paper), as well as concatenation and `replace` (with the ‘g’—global—flag, called `replaceAll` in this paper), are present.

The program analysis is to ascertain whether `urlXssSanitise` indeed works, namely, after applying `urlXssSanitise` to the input URL, “`script`” does not appear. This problem can be reduced to checking whether there is an execution path of `urlXssSanitise` that produces an output such that its query or fragment component contains occurrences of “`script`”. For instance, assuming the “if” branch is executed, we will need to solve the path feasibility of the following JavaScript program in single static assignment (SSA) form,

<MINTED>

where the `assert(cond)` statement checks that the condition *cond* is satisfied. As one will see later, this can be directly encoded in our constraint language (cf. Section ??) and handled by the decision procedure (cf. Section ??). Our solver OSTRICH+ can solve the path feasibility of the aforementioned SSA program in several seconds. This is far from trivial since the solver needs to tackle complex string functions such as `trim()` (modelled as finite transducers) and `replaceAll`, as well as `indexOf` and `substring`, which is beyond the scope of other existing decision procedures.

E.1 Formulation in $\mathbf{SL}_{\text{int}}$

After adapting the syntax of the program corresponding to the “if” branch of `urlXssSanitise(url)`, we obtain the following $\mathbf{SL}_{\text{int}}$ program,

```

assert (prothostpath  $\in \mathcal{A}_\varepsilon$ ); assert (querfrag  $\in \mathcal{A}_\varepsilon$ );
url1 :=  $\mathcal{T}_{\text{trim}}$ (url); assert (qmarkpos = indexOf7(url1, 0));
assert (sharppos = indexOf#(url1, 0)); assert (qmarkpos  $\geq 0$ );
prothostpath1 := substring(url1, 0, qmarkpos);
querfrag1 := substring(url1, qmarkpos, length(url1) - qmarkpos);
querfrag2 := replaceAllscript,  $\varepsilon$ (querfrag1);
url2 := prothostpath1 · querfrag2; assert (querfrag2  $\in \mathcal{A}_{\Sigma^* \text{script} \Sigma^*}$ )

```

where $\mathcal{A}_\varepsilon$ is the NFA defining $\{\varepsilon\}$, $\mathcal{T}_{\text{trim}}$ is an NFT to model the sanitisation operation `trim()`, and $\mathcal{A}_{\Sigma^* \text{script} \Sigma^*}$ is the NFA defining $\{w \text{script} w' \mid w, w' \in \Sigma^*\}$.

E.2 An illustration of the decision procedure

Consider the program S associated with `urlXssSanitise(url)`. We show how to decide its path feasibility.

Step I. Vacuous since S contains only atomic assertions already.

Step II. Nondeterministically choose to replace `indexOf#(url1, 0)` with `-1` and add `assert (url1  $\in \mathcal{A}_{\Sigma^* \# \Sigma^*}$ )` to S .

Step III. Replace `length(url1)` with `$i'_1$` and add `assert (url1  $\in \mathcal{A}_{\text{len}}[i'_1/r_1]$ )` to S , moreover, replace `indexOf7(url1, 0)` with `$i'_3$` and add `assert (0 =  $i'_2$ )`; `assert (url1  $\in \mathcal{A}_{\text{indexOf}}[i'_2/r_1, i'_3/r_2]$ )` to S , where `$i'_1, i'_2, i'_3$` are fresh integer variables. Then we get the following program (still denoted by S),

```

assert (prothostpath  $\in \mathcal{A}_\varepsilon$ ); assert (querfrag  $\in \mathcal{A}_\varepsilon$ ); url1 :=  $\mathcal{T}_{\text{trim}}$ (url);
assert (qmarkpos =  $i'_3$ ); assert (sharppos = -1); assert (qmarkpos  $\geq 0$ );
prothostpath1 := substring(url1, 0, qmarkpos);
querfrag1 := substring(url1, qmarkpos,  $i'_1$  - qmarkpos);
querfrag2 := replaceAllscript,  $\varepsilon$ (querfrag1);
url2 := prothostpath1 · querfrag2; assert (querfrag2  $\in \mathcal{A}_{\Sigma^* \text{script} \Sigma^*}$ );
assert (url1  $\in \mathcal{A}_{\Sigma^* \# \Sigma^*}$ ); assert (url1  $\in \mathcal{A}_{\text{len}}[i'_1/r_1]$ );
assert (0 =  $i'_2$ ); assert (url1  $\in \mathcal{A}_{\text{indexOf}}[i'_2/r_1, i'_3/r_2]$ ).

```

Step IV. Since there is no CEFA constraints for `url2`, removing the last assignment statement of S , i.e. `url2 := prothostpath1 · querfrag2`, is easy and in this case we add no statements to S . After this, `querfrag2 := replaceAllscript,  $\varepsilon$ (querfrag1)` becomes the last assignment statement. Suppose $\mathcal{A}' = (Q', \Sigma, \delta', I', F')$ is an NFA representing $(\text{replaceAll}_{\text{script}, \varepsilon})_0^{-1}(\mathcal{L}(\mathcal{A}_{\Sigma^* \text{script} \Sigma^*}))$, namely, the pre-image of $\mathcal{L}(\mathcal{A}_{\Sigma^* \text{script} \Sigma^*})$ under `replaceAllscript,  $\varepsilon$` . Then we remove this assignment

statement and add $\text{assert}(\text{querfrag1} \in \mathcal{A}')$, resulting into the following program

```

assert (prothostpath  $\in \mathcal{A}_\varepsilon$ ); assert (querfrag  $\in \mathcal{A}_\varepsilon$ ); url1 :=  $\mathcal{T}_{\text{trim}}(\text{url1})$ ;
assert (qmarkpos =  $i'_3$ ); assert (sharppos = -1); assert (qmarkpos  $\geq 0$ );
prothostpath1 := substring(url1, 0, qmarkpos);
querfrag1 := substring(url1, qmarkpos,  $i'_1 - \text{qmarkpos}$ );
assert (querfrag2  $\in \mathcal{A}_{\Sigma^* \text{script} \Sigma^*}$ ); assert (url1  $\in \mathcal{A}_{\Sigma^* \# \Sigma^*}$ );
assert (url1  $\in \mathcal{A}_{\text{len}[i'_1/r_1]}$ ); assert ( $0 = i'_2$ );
assert (url1  $\in \mathcal{A}_{\text{indexOf}[i'_2/r_1, i'_3/r_2]}$ ); assert (querfrag1  $\in \mathcal{A}'$ ).

```

From Example ??, we know that $\text{substring}_0^{-1}(\mathcal{L}(\mathcal{A}'))$ can be represented by some CEFA $\mathcal{B} = (Q'', R'', \delta'', I'', F'')$ with $Q'' = Q' \times \{p_0, p_1, p_2\}$ and $R'' = (r'_{1,1}, r'_{1,2})$ (where $r'_{1,1}$ and $r'_{1,2}$ are fresh integer variables). Then we remove $\text{querfrag1} := \text{substring}(\text{url1}, \text{qmarkpos}, i'_1 - \text{qmarkpos})$, add $\text{assert}(\text{url1} \in \mathcal{B})$; $\text{assert}(r'_{1,1} = \text{qmarkpos})$; $\text{assert}(r'_{1,2} = i'_1 - \text{qmarkpos})$, and get the following program

```

assert (prothostpath  $\in \mathcal{A}_\varepsilon$ ); assert (querfrag  $\in \mathcal{A}_\varepsilon$ ); url1 :=  $\mathcal{T}_{\text{trim}}(\text{url1})$ ;
assert (qmarkpos =  $i'_3$ ); assert (sharppos = -1); assert (qmarkpos  $\geq 0$ );
prothostpath1 := substring(url1, 0, qmarkpos);
assert (querfrag2  $\in \mathcal{A}_{\Sigma^* \text{script} \Sigma^*}$ ); assert (url1  $\in \mathcal{A}_{\Sigma^* \# \Sigma^*}$ );
assert (url1  $\in \mathcal{A}_{\text{len}[i'_1/r_1]}$ ); assert ( $0 = i'_2$ );
assert (url1  $\in \mathcal{A}_{\text{indexOf}[i'_2/r_1, i'_3/r_2]}$ ); assert (querfrag1  $\in \mathcal{A}'$ );
assert (url1  $\in \mathcal{B}$ ); assert ( $r'_{1,1} = \text{qmarkpos}$ ); assert ( $r'_{1,2} = i'_1 - \text{qmarkpos}$ ).

```

We can continue the process until the problem contains no assignment statement.

Step V. Straightforward by utilising Proposition ??.

F Implementation

OSTRICH+ performs a depth-first exploration of the search tree resulting from repeatedly splitting the disjunctions (or unions) in the cost-enriched recognisable pre-images of CERLs under string functions, as well as the case splits in the semantics of `indexOf` and `substring`. The pseudo-code of Step II-III of the decision procedure is given by the function *checkSat* in Algorithm ??, which calls two functions *indexofCaseSplit* in Algorithm ?? and *substringCaseSplit* in Algorithm ?? for the case splits in the semantics of `indexOfv` and `substring` respectively. Moreover, *checkSat* calls a recursive function *BackDfsExp* in Algorithm ?? for the depth-first exploration (Step IV of the decision procedure), which in turn calls a function *CheckCefaLIASat* to solve the $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem (Step V). Note that Step I of the decision procedure is handled by the DPLL(T) procedure in Princess and is omitted here.

Algorithm 1: Function *checkSat* for Step II-III

Input: *active*: set of CEFA constraints, *arith*: arithmetic constraints, *funApps*: acyclic set of assignment statements.
 Result: *sat* if the input constraints are satisfiable, and *unsat* otherwise.

```

1 for each partition  $(\mathcal{I}_l)_{l \in [5]}$  of the set of  $\text{indexOf}_v(x, i)$  in arith and
   each partition  $(\mathcal{J}_l)_{l \in [3]}$  of the set of  $\text{substring}(x, i, j)$  in funApps /* the
   partitions refer to (1)-(5) for  $\text{indexOf}_v(x, i)$  and (1)-(3) for  $\text{substring}(x, i, j)$  in
   Step II of Section ?? */
2 do
   /* Case splits for semantics of  $\text{indexOf}$  and  $\text{substring}$  */
3    $(\text{active}, \text{arith}, \text{funApps}) = \text{indexofCaseSplit}(\text{active}, \text{arith}, \text{funApps}, (\mathcal{I}_l)_{l \in [5]});$ 
4    $(\text{active}, \text{arith}, \text{funApps}) = \text{substringCaseSplit}(\text{active}, \text{arith}, \text{funApps}, (\mathcal{J}_l)_{l \in [3]});$ 
5   for each  $\text{length}(x)$  occurring in arith do
6     choose a fresh integer variable  $i$ ;
7      $\text{active} \leftarrow \text{active} \cup \{x \in \mathcal{A}_{\text{len}}[i/r_1]\}; \text{arith} \leftarrow \text{arith}[i/\text{length}(x)];$ 
8     for each  $\text{indexOf}_v(x, i)$  occurring in arith do
9       choose fresh integer variables  $i_1, i_2$ ;
10       $\text{active} \leftarrow \text{active} \cup \{x \in \mathcal{A}_{\text{indexOf}_v}[i_1/r_1, i_2/r_2]\};$ 
11       $\text{arith} \leftarrow \text{arith}[i_2/\text{indexOf}_v(x, i)] \wedge i = i_1$ ;
12      if BackDfsExp(active,  $\emptyset$ , arith, funApps) then
13        return sat;
14 return unsat;
```

Algorithm 2: *indexofCaseSplit* for case splits in the semantics of indexOf_v

Input: *active*: set of CEFA constraints, *arith*: arithmetic constraint, *funApps*: acyclic set of assignment statements, and $(\mathcal{I}_l)_{l \in [5]}$: subsets of $\text{indexOf}_v(x, i)$ string terms
 Result: $(\text{active}, \text{arith}, \text{funApps})$

```

1 for each  $\text{indexOf}_v(x, i) \in \mathcal{I}_1$  do
2    $\text{arith} \leftarrow \text{arith}[\text{indexOf}_v(x, 0)/\text{indexOf}_v(x, i)] \wedge i < 0$ ;
3 for each  $\text{indexOf}_v(x, i) \in \mathcal{I}_2$  do
4    $\text{active} \leftarrow \text{active} \cup \{x \in \mathcal{A}_{\Sigma^*, v, \Sigma^*}\};$ 
5    $\text{arith} \leftarrow \text{arith}[-1/\text{indexOf}_v(x, i)] \wedge i < 0$ ;
6 for each  $\text{indexOf}_v(x, i) \in \mathcal{I}_3$  do
7    $\text{arith} \leftarrow \text{arith}[-1/\text{indexOf}_v(x, i)] \wedge i \geq \text{length}(x)$ ;
8 for each  $\text{indexOf}_v(x, i) \in \mathcal{I}_4$  do
9    $\text{arith} \leftarrow \text{arith}[-1/\text{indexOf}_v(x, i)] \wedge i \geq 0 \wedge i < \text{length}(x)$ ;
10 for each  $\text{indexOf}_v(x, i) \in \mathcal{I}_5$  do
11   choose fresh variables  $y$  and  $j$ ;
12    $\text{active} \leftarrow \text{active} \cup \{y \in \mathcal{A}_{\Sigma^*, v, \Sigma^*}\};$ 
13    $\text{arith} \leftarrow \text{arith}[-1/\text{indexOf}_v(x, i)] \wedge i \geq 0 \wedge i < \text{length}(x) \wedge j = \text{length}(x) - i$ ;
14    $\text{funApps} \leftarrow \text{funApps} \cup \{y := \text{substring}(x, i, j)\};$ 
```

Algorithm 3: *substringCaseSplit* for case splits in the semantics of *substring*

Input: *active*: set of CEFA constraints, *arith*: arithmetic constraint, *funApps*: acyclic set of assignment statements, and $(\mathcal{I}_l)_{l \in [5]}$: subsets of $\text{indexOf}_v(x, i)$ string terms
 Result: $(\text{active}, \text{arith}, \text{funApps})$

```

1 for each  $y := \text{substring}(x, i, j) \in \mathcal{J}_1$  do
2    $\text{arith} \leftarrow \text{arith} \wedge i \geq 0 \wedge i + j \leq \text{length}(x)$ ;
3 for each  $y := \text{substring}(x, i, j) \in \mathcal{J}_2$  do
4   choose a fresh integer variable  $i'$ ;
5    $\text{arith} \leftarrow \text{arith} \wedge i \geq 0 \wedge i \leq \text{length}(x) \wedge i + j > \text{length}(x) \wedge i' = \text{length}(x) - i$ ;
6    $\text{funApps} \leftarrow \text{funApps}[y := \text{substring}(x, i, i')/y := \text{substring}(x, i, j)]$ ;
7 for each  $y := \text{substring}(x, i, j) \in \mathcal{J}_3$  do
8    $\text{arith} \leftarrow \text{arith} \wedge i < 0$ ;
9    $\text{active} \leftarrow \text{active} \cup \{y \in \mathcal{A}_e\}$ ;
10   $\text{funApps} \leftarrow \text{funApps} \setminus \{y := \text{substring}(x, i, j)\}$ ;
```

Optimisations for solving the $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem. From Proposition ??, a natural approach to solve the $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem is to compute an existential LIA formula defining the Parikh image of products of CEFAs, and then use off-the-shelf SMT solvers (e.g. CVC4 or Z3) to decide the satisfiability of the existential LIA formula. However, our preliminary experiments show that this approach suffers from a scalability issue, in particular, the state-space explosion when computing products of CEFAs. In the implementation of the function *CheckCefaLIASat* in Algorithm ??, we opt to utilise the symbolic model checker nuXmv [?] to mitigate the state-space explosion during the computation of products of CEFAs. The nuXmv tool is a well-known symbolic model checker that is capable of analysing both finite and infinite state systems. Our technique is to encode $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ as an instance of the model checking problem, which can be solved by nuXmv. Since $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ is a problem for quantifier-free LIA formulas and CEFAs that contain integer variables, the $\text{SAT}_{\text{CEFA}}[\text{LIA}]$ problem actually corresponds to the problem of model checking infinite state systems.

Algorithm 4: Function *BackDfsExp* for Step IV (depth-first exploration)

Input: *active*, *passive*: sets of CEFA constraints, *arith*: arithmetic constraints,
funApps: acyclic set of assignment statements.
Result: *sat* if the input constraints are satisfiable, and *unsat* otherwise.

```

1 if active =  $\emptyset$  then
    /* Check whether the LIA constraint arith is satisfiable with respect to the
       CEFA constraints in passive (i.e. Step V). */
2     return CheckCefaLIASat(passive, arith);
3 else
4     choose a CEFA constraint  $x \in \mathcal{A}$  in active with  $R(\mathcal{A}) = (r_1, \dots, r_k)$ ;
5     if there is an assignment  $x := f(y_1, \vec{i}_1, \dots, y_l, \vec{i}_l)$  defining  $x$  in funApps with
         $\vec{i}_j = (i_{j,1}, \dots, i_{j,k_j})$  for  $j \in [l]$  then
6         compute  $f_{R(\mathcal{A})}^{-1}(\mathcal{L}(\mathcal{A})) = ((\mathcal{A}_j^{(1)}, \dots, \mathcal{A}_j^{(l)})_{j \in [n]}, \vec{t})$  where
             $R(\mathcal{A}_j^{(j')}) = ((r')^{(j',1)}, \dots, (r')^{(j',k_{j'})}, r_1^{(j')}, \dots, r_k^{(j')})$  for  $j \in [n]$  and  $j' \in [l]$ ;
7         active  $\leftarrow$  active  $\setminus \{x \in \mathcal{A}\}$ ; passive  $\leftarrow$  passive  $\cup \{x \in \mathcal{A}\}$ ;
8         for  $j \leftarrow 1$  to  $n$  do
9             active  $\leftarrow$  active  $\cup \{y_1 \in \mathcal{A}_j^{(1)}, \dots, y_l \in \mathcal{A}_j^{(l)}\}$ ;
10            arith  $\leftarrow$  arith  $\wedge \bigwedge_{j' \in [l], j'' \in [k_{j'}]} i_{j',j''} = (r')^{(j',j'')} \wedge \bigwedge_{j' \in [k]} r_{j'} = t_{j'}$ ;
11            if active  $\cup$  passive is inconsistent then
12                continue ; /* backtrack */
13            else
14                switch BackDfsExp(active, passive, arith, funApps) do
15                    case sat do return sat;
16                    case unsat do
17                        continue ; /* backtrack */
18            return unsat;
19 else
20     return BackDfsExp(active  $\setminus \{x \in \mathcal{A}\}$ , passive  $\cup \{x \in \mathcal{A}\}$ , arith, funApps);

```
